Pipeline de Integração Contínua – TeVejo

Repositório: `tads-cnat/tevejo` · Ferramenta: **GitHub Actions** · Registry: **Docker Hub**

Parte	Workflow	Dockerfile	Imagem no Docker Hub
Frontend	`.github/workflows/frontend-ci.yml`	`src/frontend-angular/Dockerfile`	`<usuario>/tevejo-frontend`
Backend	`.github/workflows/backend-ci.yml`	`src/backend_django/Dockerfile`	`<usuario>/tevejo-backend`

As credenciais do Docker Hub ficam nos secrets `DOCKERHUB_USERNAME` e `DOCKERHUB_TOKEN` do repositório.

1. Frontend (Angular)

Uma única job (`frontend-pipeline`), executada em `src/frontend-angular`:

1. **Checkout do código** – `actions/checkout@v4`.
2. **Configuração do projeto** – `actions/setup-node@v4` com Node 24 e cache do npm; `npm ci`.
3. **Testes** – `npm run test -- --watch=false`.
4. **Build do software** – `npm run build` (gera o bundle em `dist/`).
5. **Login no Docker Hub** – `docker/login-action@v3`.
6. **Build da imagem** – `docker build -t <usuario>/tevejo-frontend:latest .`
7. **Push da imagem** – `docker push <usuario>/tevejo-frontend:latest`.

Dockerfile: base `node:24-alpine`, instala as dependências, copia o código, roda `npm run build` e sobe a aplicação na porta 4200.

2. Backend (Django)

Duas jobs encadeadas. A segunda (`docker`) só roda se a primeira passar (`needs: build-e-testes`) e não roda em pull requests, para não publicar imagem de código que ainda não foi aceito.

Job `build-e-testes` – build de software

1. **Checkout do código**.
2. **Configuração do projeto** – `actions/setup-python@v5` com Python 3.12 e cache do pip; `pip install -r requirements.txt`.
3. **Build** – `python manage.py check` (valida settings, apps, URLs e models) e `python -m compileall` (compila todos os `.py`; um erro de sintaxe derruba o pipeline aqui).
4. **Testes** – `python manage.py test` (5 testes em `usuarios/tests.py`: cadastro válido, senha fraca, login correto, login com senha errada e rota `/me/` protegida).
5. **Artefato** – o código validado é empacotado em `tevejo-backend.tar.gz` e publicado com `actions/upload-artifact@v4` (equivale ao `.jar` de um projeto Java).

Job `docker` – build de ambiente

1. **Login no Docker Hub**.
2. **Build da imagem** com duas tags: `latest` e o SHA do commit (`github.sha`), o que permite saber exatamente de qual commit veio cada imagem.
3. **Push** das duas tags para o Docker Hub.

Dockerfile: base `python:3.12-slim`; copia primeiro o `requirements.txt` e instala as dependências (camada reaproveitada pelo cache enquanto as dependências não mudam), depois copia o código. Ao iniciar, roda `migrate` e sobe o servidor na porta 8000. O `.dockerignore` deixa de fora `__pycache__`, banco SQLite, `media/` e ambientes virtuais. O banco de dados não entra no pipeline.

Para rodar a imagem localmente:

```
docker run -p 8000:8000 <usuario>/tevejo-backend:latest
```

3. Falhas encontradas e correções

Falha 1 – Backend: dependência ausente (`requests`). Causa: `usuarios/serializers.py` importa `google.auth.transport.requests`, mas o `requirements.txt` não listava `requests`. Em um ambiente limpo (CI ou container), o `manage.py check` falhava com `ImportError: The requests library is not installed`. Correção: `requests` foi adicionado ao `requirements.txt`. Funcionava na máquina de quem desenvolveu só porque o pacote já estava instalado lá.

Falha 2 – Frontend: versão do Node. Causa: o workflow e o Dockerfile usavam Node 20, versão que o Angular 21 do projeto não aceita, e o pipeline falhava. Correção: Node 24 no `setup-node` e na imagem base (de `node:20-alpine` para `node:24-alpine`).

Falha 3 – Frontend: testes unitários quebrados. Causa: os specs (`app.spec.ts`, `conteudo-educacional.spec.ts`, `criar-comunidade.spec.ts`) importavam os componentes com nomes antigos (`App`, `ConteudoEducacional`) e não registravam os providers de que eles dependem (`provideRouter`, `provideHttpClient`), e a etapa de testes falhava. Correção: imports trocados para `AppComponent` / `ConteudoEducacionalComponent` e providers adicionados no `TestBed` de cada spec.

> Prints a incluir no PDF de entrega: pipeline executando com sucesso (aba *Actions*), build Docker concluído (log da etapa de build/push ou página da imagem no Docker Hub) e a execução que falhou.

4. Respostas

Qual a diferença entre build de software e build de ambiente? O build de software transforma o código-fonte em um artefato executável e verificado (`.jar`, bundle `dist/` do Angular, pacote do backend). O build de ambiente gera uma imagem de container com tudo o que é necessário para executar esse artefato: sistema base, runtime (Node/Python), dependências e configuração. O primeiro responde "o código está correto?"; o segundo, "onde e como ele roda?".

Por que o teste deve ocorrer antes do build da imagem? Para não empacotar nem publicar código com defeito. Se os testes falharem, o pipeline para antes de gastar tempo com o build da imagem e evita que uma imagem quebrada chegue ao Docker Hub com a tag `latest`. No backend isso é garantido pelo `needs`: a job `docker` só começa se a job de testes passar.

Qual etapa do pipeline falharia caso o código não compilasse? A etapa de build do software: `npm run build` no frontend e `manage.py check` / `compileall` no backend. Como as etapas seguintes dependem dela, testes, build da imagem e push nem chegam a executar.

Que vantagem existe em automatizar esse processo? Cada push é validado da mesma forma, em um ambiente limpo, sem depender da máquina de ninguém (a falha do `requests` foi achada justamente assim). Os erros aparecem cedo, a imagem publicada é sempre rastreável até um commit, e o time não perde tempo repetindo build, testes e publicação manualmente.